Jeevan Raksha Pharmacy

End-to-End MySQL Project

Schema • Sample Data • Indexes • Views • Stored Procedures • Triggers • Query Bank with Answers

Database: jeevan_raksha_pharmacy | RDBMS: MySQL

Table of Contents

- PART 1: DATABASE & TABLE CREATION
- PART 2: SAMPLE DATA
- PART 3: INDEXES
- PART 4: VIEWS
- PART 5: STORED PROCEDURES
- PART 6: TRIGGERS
- PART 7: QUERY BANK - LEVEL 1 : SIMPLE QUERIES (Original Q1-Q4)
- PART 8: QUERY BANK - LEVEL 2 : INTERMEDIATE QUERIES (Original Q5-Q8)
- PART 9: QUERY BANK - LEVEL 3 : COMPLEX QUERIES (Original Q9-Q11)
- PART 10: QUERY BANK - NEW QUERIES (Q12-Q21)
- PART 11: DEMO CALLS - Views / Stored Procedures / Triggers

Project Overview

JEEVAN RAKSHA PHARMACY | END-TO-END MYSQL PROJECT

RDBMS : MySQL 8.0+ (also compatible with MariaDB 10.x)

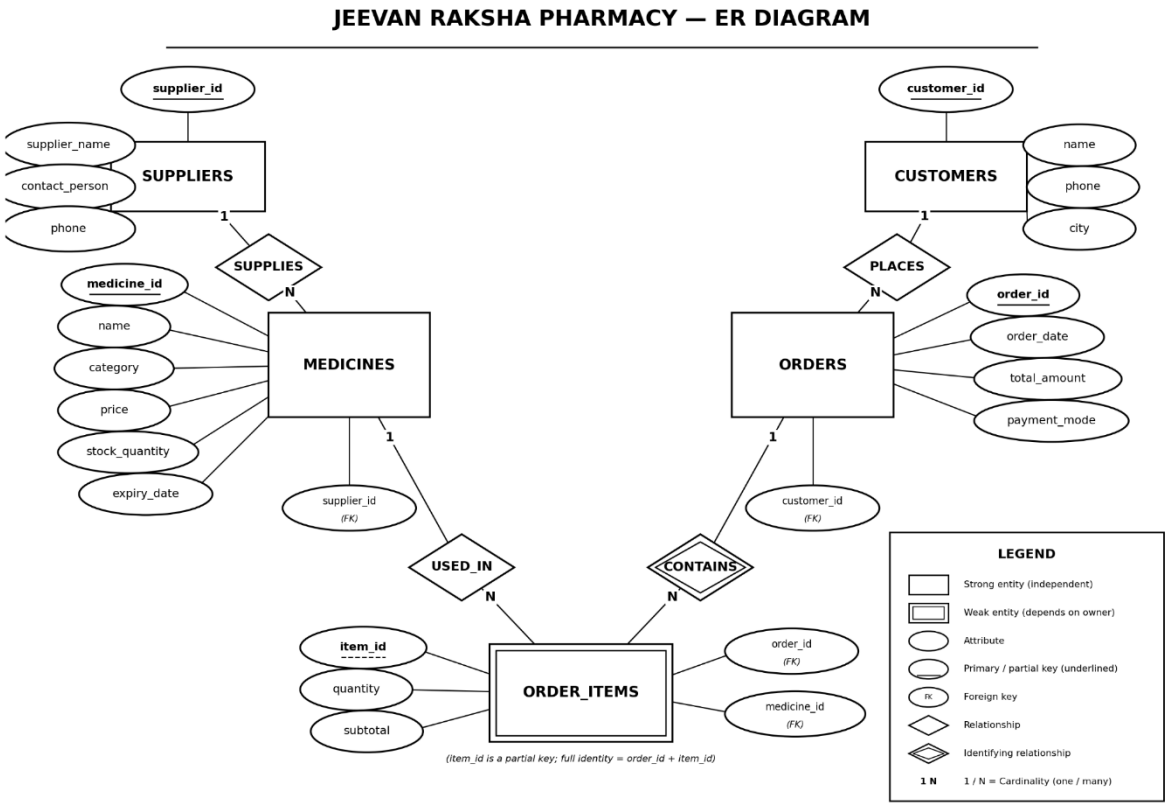
Notes : Every ANSWER below each query is a comment showing what

that query returns when run against the sample data in

Contents:

- PART 1 : Database & Table Creation (schema)

- PART 2 : Sample Data (INSERT statements)
- PART 3 : Indexes
- PART 4 : Views
- PART 5 : Stored Procedures
- PART 6 : Triggers (+ supporting stock_log table)
- PART 7 : Query Bank - Level 1 (Original Q1-Q4) + Answers
- PART 8 : Query Bank - Level 2 (Original Q5-Q8) + Answers
- PART 9 : Query Bank - Level 3 (Original Q9-Q11) + Answers
- PART 10 : Query Bank - New Queries (Q12-Q21) + Answers
- PART 11 : Demo calls for Views / Procedures / Triggers
- PART 2. Run the SELECT yourself to confirm.



DATABASE & TABLE CREATION

```
DROP DATABASE IF EXISTS jeevan_raksha_pharmacy;
CREATE DATABASE jeevan_raksha_pharmacy;
USE jeevan_raksha_pharmacy;

-- ---------- Strong Entity: CUSTOMERS ----------
CREATE TABLE customers (
        customer_id INT AUTO_INCREMENT PRIMARY KEY,
        name       VARCHAR(100),
        phone      VARCHAR(15),
        city       VARCHAR(50)
);

-- ---------- Strong Entity: SUPPLIERS ----------
CREATE TABLE suppliers (
        supplier_id      INT AUTO_INCREMENT PRIMARY KEY,
        supplier_name  VARCHAR(100),
        contact_person VARCHAR(100),
        phone            VARCHAR(15)
);

-- ---------- Strong Entity: MEDICINES ----------
CREATE TABLE medicines (
        medicine_id    INT AUTO_INCREMENT PRIMARY KEY,
        name           VARCHAR(100),
        category       VARCHAR(50),
        price          DECIMAL(10,2),
        stock_quantity INT,
```

```
    expiry_date     DATE,
    supplier_id     INT,
    FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
);

-- ---------- Strong Entity: ORDERS (Bills) ----------
CREATE TABLE orders (
    order_id     INT AUTO_INCREMENT PRIMARY KEY,
    customer_id  INT,
    order_date   DATE,
    total_amount DECIMAL(10,2),
    payment_mode ENUM('UPI', 'Cash', 'Card'),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- ---------- Weak Entity: ORDER_ITEMS ----------
-- Cannot exist without a parent ORDER (identifying relationship).
-- True identity of a row = order_id (owner key) + item_id (partial key).
CREATE TABLE order_items (
    item_id     INT AUTO_INCREMENT PRIMARY KEY,
    order_id    INT,
    medicine_id INT,
    quantity    INT,
    subtotal    DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id)
);
```

SAMPLE DATA

```
-- Customers
INSERT INTO customers (name, phone, city) VALUES
('Rahul Sharma', '9876543210', 'Mumbai'),
('Priya Verma',  '9123456789', 'Delhi'),
('Amit Patel',   '9988776655', 'Ahmedabad'),
('Sneha Reddy',  '8877665544', 'Hyderabad'),
('Vikram Singh', '7766554433', 'Mumbai');

-- Suppliers
INSERT INTO suppliers (supplier_name, contact_person, phone) VALUES
('Apollo Distributors',   'Rajesh Gupta', '022-123456'),
('MedPlus Supply Chain',  'Suresh Nair',  '040-987654'),
('Himalaya Wellness',  'Anjali Mehta', '011-456789');

-- Medicines
INSERT INTO medicines (name, category, price, stock_quantity, expiry_date, supplier_id)
VALUES
('Dolo 650',      'Tablet',          30.00,  500, '2025-12-31', 1),
('Azithral 500', 'Tablet',   120.00,   50, '2024-05-20', 1),
('Benadryl',      'Syrup',    110.00,   20, '2024-11-15', 2),
('Combiflam',   'Tablet',    45.00,  200, '2026-01-01', 3),
```

```
('Insulin Pen',  'Injection',800.00,        5, '2024-03-10', 2);


-- Orders (Bills)
INSERT INTO orders (customer_id, order_date, total_amount, payment_mode) VALUES
(1, '2023-10-01', 150.00,  'UPI'),
(2, '2023-10-02', 240.00,  'Card'),
(3, '2023-10-05',  45.00,  'Cash'),
(1, '2023-10-10', 800.00,  'UPI'),
(4, '2023-10-12',1200.00,  'Card');


-- Order Items
INSERT INTO order_items (order_id, medicine_id, quantity, subtotal) VALUES
(1, 1, 5, 150.00),  -- Rahul bought 5 Dolo 650
(2, 2, 2, 240.00),  -- Priya bought 2 Azithral 500
(3, 4, 1,  45.00),  -- Amit  bought 1 Combiflam
(4, 5, 1, 800.00),  -- Rahul bought 1 Insulin Pen
(5, 5, 1, 800.00),  -- Sneha bought 1 Insulin Pen
(5, 2, 2, 240.00),  -- Sneha bought 2 Azithral 500
(5, 1, 5, 150.00);  -- Sneha bought 5 Dolo 650
```

INDEXES

```
-- Foreign-key / frequently-filtered columns get explicit indexes to
-- speed up the JOINs and WHERE clauses used throughout this project.

CREATE INDEX idx_medicines_supplier   ON medicines(supplier_id);
CREATE INDEX idx_orders_customer          ON orders(customer_id);
CREATE INDEX idx_orderitems_order         ON order_items(order_id);
```

```
CREATE INDEX idx_orderitems_medicine  ON order_items(medicine_id);
CREATE INDEX idx_medicines_category   ON medicines(category);
CREATE INDEX idx_medicines_expiry     ON medicines(expiry_date);
CREATE INDEX idx_customers_city       ON customers(city);
CREATE INDEX idx_orders_paymentmode   ON orders(payment_mode);
CREATE INDEX idx_orders_date          ON orders(order_date);

-- Verify indexes:
-- SHOW INDEX FROM medicines;
-- SHOW INDEX FROM orders;
-- SHOW INDEX FROM order_items;
```

VIEWS

```
-- 4.1 Flattened order + customer info (for reporting/UI)
CREATE OR REPLACE VIEW view_order_details AS
SELECT
        o.order_id,
        c.name          AS customer_name,
        c.city          AS customer_city,
        o.order_date,
        o.total_amount,
        o.payment_mode
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;

-- 4.2 Medicines that need reordering (stock below 50 units)
```

```
CREATE OR REPLACE VIEW view_low_stock AS
SELECT
        m.medicine_id,
        m.name,
        m.stock_quantity,
        s.supplier_name
FROM medicines m
JOIN suppliers s ON m.supplier_id = s.supplier_id
WHERE m.stock_quantity < 50;

-- 4.3 Lifetime spend per customer
CREATE OR REPLACE VIEW view_customer_spend AS
SELECT
        c.customer_id,
        c.name,
        COUNT(o.order_id)              AS total_orders,
        COALESCE(SUM(o.total_amount), 0)  AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;

-- 4.4 Best-seller ranking (quantity sold per medicine)
CREATE OR REPLACE VIEW view_medicine_sales AS
SELECT
        m.medicine_id,
        m.name,
        COALESCE(SUM(oi.quantity), 0) AS total_qty_sold
FROM medicines m
```

```
LEFT JOIN order_items oi ON m.medicine_id = oi.medicine_id
GROUP BY m.medicine_id, m.name;
```

STORED PROCEDURES

```
-- 5.1 All orders placed by one customer
DROP PROCEDURE IF EXISTS sp_get_customer_orders;
DELIMITER $$
CREATE PROCEDURE sp_get_customer_orders(IN p_customer_id INT)
BEGIN
        SELECT o.order_id, o.order_date, o.total_amount, o.payment_mode
        FROM orders o
        WHERE o.customer_id = p_customer_id
        ORDER BY o.order_date;
END $$
DELIMITER ;

-- 5.2 Place a new order for ONE medicine (transactional, stock-checked)
--      Creates the order + order_item; the trg_reduce_stock trigger
--      (PART 6) automatically decrements medicines.stock_quantity.
DROP PROCEDURE IF EXISTS sp_place_order;
DELIMITER $$
CREATE PROCEDURE sp_place_order(
        IN p_customer_id  INT,
        IN p_medicine_id  INT,
        IN p_quantity  INT,
        IN p_payment_mode VARCHAR(10)
```

```
)
BEGIN
        DECLARE v_price        DECIMAL(10,2);
        DECLARE v_stock        INT;
        DECLARE v_subtotal DECIMAL(10,2);
        DECLARE v_order_id INT;

        SELECT price, stock_quantity INTO v_price, v_stock
        FROM medicines
        WHERE medicine_id = p_medicine_id
        FOR UPDATE;

        IF v_stock < p_quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock for this medicine';
        ELSE
        START TRANSACTION;
        SET v_subtotal = v_price * p_quantity;

        INSERT INTO orders (customer_id, order_date, total_amount, payment_mode)
        VALUES (p_customer_id, CURDATE(), v_subtotal, p_payment_mode);
        SET v_order_id = LAST_INSERT_ID();

        INSERT INTO order_items (order_id, medicine_id, quantity, subtotal)
        VALUES (v_order_id, p_medicine_id, p_quantity, v_subtotal);

        COMMIT;
        END IF;
```

```
END $$
DELIMITER ;

-- 5.3 Revenue report grouped by payment mode
DROP PROCEDURE IF EXISTS sp_revenue_by_mode;
DELIMITER $$
CREATE PROCEDURE sp_revenue_by_mode()
BEGIN
        SELECT payment_mode,
        SUM(total_amount) AS total_revenue,
        COUNT(*)       AS order_count
        FROM orders
        GROUP BY payment_mode;
END $$
DELIMITER ;

-- 5.4 Restock a medicine by a given quantity (logged automatically by
--      trg_restock_log in PART 6)
DROP PROCEDURE IF EXISTS sp_restock_medicine;
DELIMITER $$
CREATE PROCEDURE sp_restock_medicine(
        IN p_medicine_id INT,
        IN p_add_qty  INT
)
BEGIN
        UPDATE medicines
        SET stock_quantity = stock_quantity + p_add_qty
        WHERE medicine_id = p_medicine_id;
```

```
END $$
DELIMITER ;
```

TRIGGERS

```
-- 6.0 Supporting audit table for stock changes
CREATE TABLE stock_log (
        log_id      INT AUTO_INCREMENT PRIMARY KEY,
        medicine_id INT,
        change_qty  INT,
        change_type VARCHAR(20),
        change_date DATETIME DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id)
);

-- 6.1 Reduce stock automatically whenever an order_item is inserted
DROP TRIGGER IF EXISTS trg_reduce_stock;
DELIMITER $$
CREATE TRIGGER trg_reduce_stock
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
        UPDATE medicines
        SET stock_quantity = stock_quantity - NEW.quantity
        WHERE medicine_id = NEW.medicine_id;

        INSERT INTO stock_log (medicine_id, change_qty, change_type)
```

```
        VALUES (NEW.medicine_id, -NEW.quantity, 'SALE');
END $$
DELIMITER ;

-- 6.2 Block an order_item if it would oversell the available stock
DROP TRIGGER IF EXISTS trg_prevent_negative_stock;
DELIMITER $$
CREATE TRIGGER trg_prevent_negative_stock
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
        DECLARE v_stock INT;
        SELECT stock_quantity INTO v_stock
        FROM medicines WHERE medicine_id = NEW.medicine_id;

        IF v_stock < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot place order: insufficient stock';
        END IF;
END $$
DELIMITER ;

-- 6.3 Log every manual restock (stock_quantity increased on medicines)
DROP TRIGGER IF EXISTS trg_restock_log;
DELIMITER $$
CREATE TRIGGER trg_restock_log
AFTER UPDATE ON medicines
FOR EACH ROW
```

```
BEGIN
        IF NEW.stock_quantity > OLD.stock_quantity THEN
        INSERT INTO stock_log (medicine_id, change_qty, change_type)
        VALUES (NEW.medicine_id, NEW.stock_quantity - OLD.stock_quantity, 'RESTOCK');
        END IF;
END $$
DELIMITER ;

-- 6.4 Prevent deleting a supplier that still has medicines linked to it
DROP TRIGGER IF EXISTS trg_block_supplier_delete;
DELIMITER $$
CREATE TRIGGER trg_block_supplier_delete
BEFORE DELETE ON suppliers
FOR EACH ROW
BEGIN
        DECLARE v_count INT;
        SELECT COUNT(*) INTO v_count FROM medicines WHERE supplier_id =
OLD.supplier_id;
        IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete supplier: medicines still reference it';
        END IF;
END $$
DELIMITER ;
```

QUERY BANK - LEVEL 1 : SIMPLE QUERIES (Original Q1-Q4)

```
-- Q1. Customer List: Retrieve the name and city of all customers.
SELECT name, city FROM customers;
-- ANSWER:
--   Rahul Sharma - Mumbai
--   Priya Verma  - Delhi
--   Amit Patel   - Ahmedabad
--   Sneha Reddy  - Hyderabad
--   Vikram Singh - Mumbai

-- Q2. Stock Check: Find the names of medicines that are 'Syrup' or 'Injection'.
SELECT name FROM medicines WHERE category IN ('Syrup', 'Injection');
-- ANSWER:
--   Benadryl (Syrup)
--   Insulin Pen (Injection)

-- Q3. High Value Bills: List all orders where the total_amount is greater than 500.
SELECT * FROM orders WHERE total_amount > 500;
-- ANSWER:
--   order_id 4 - Rahul Sharma - 800.00  - UPI
--   order_id 5 - Sneha Reddy  - 1200.00 - Card

-- Q4. Mumbai Residents: Find the phone numbers of all customers living in 'Mumbai'.
SELECT name, phone FROM customers WHERE city = 'Mumbai';
-- ANSWER:
--   Rahul Sharma - 9876543210
--   Vikram Singh - 7766554433
```

QUERY BANK - LEVEL 2 : INTERMEDIATE QUERIES (Original Q5-Q8)

```
-- Q5. Payment Analysis: How many orders were paid using 'UPI'?
SELECT COUNT(*) AS upi_orders FROM orders WHERE payment_mode = 'UPI';
-- ANSWER: 2  (order_id 1 and order_id 4)

-- Q6. Supplier Count: Count how many medicines are supplied by 'Apollo Distributors'.
SELECT COUNT(*) AS medicine_count
FROM medicines m
JOIN suppliers s ON m.supplier_id = s.supplier_id
WHERE s.supplier_name = 'Apollo Distributors';
-- (Subquery version, as also allowed by the exercise)
SELECT COUNT(*) AS medicine_count
FROM medicines
WHERE supplier_id = (SELECT supplier_id FROM suppliers WHERE supplier_name = 'Apollo Distributors');
-- ANSWER: 2  (Dolo 650, Azithral 500)

-- Q7. Order Details: List the Customer Name, Order Date, and Total Amount for every order.
SELECT c.name AS customer_name, o.order_date, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date;
-- ANSWER:
--   Rahul Sharma - 2023-10-01 - 150.00
--   Priya Verma  - 2023-10-02 - 240.00
--   Amit Patel   - 2023-10-05 - 45.00
--   Rahul Sharma - 2023-10-10 - 800.00
```

-- Sneha Reddy - 2023-10-12 - 1200.00

-- Q8. Revenue by Mode: Total money earned through each payment mode.

```
SELECT payment_mode, SUM(total_amount) AS revenue
FROM orders
GROUP BY payment_mode;
```

-- ANSWER:

-- UPI : 950.00 (150 + 800)

-- Card : 1440.00 (240 + 1200)

-- Cash : 45.00

QUERY BANK - LEVEL 3 : COMPLEX QUERIES (Original Q9-Q11)

-- Q9. Best Selling Medicine: which medicine sold the most (by total quantity)?

```
SELECT m.name, SUM(oi.quantity) AS total_qty
FROM order_items oi
JOIN medicines m ON oi.medicine_id = m.medicine_id
GROUP BY m.name
ORDER BY total_qty DESC
LIMIT 1;
```

-- ANSWER: Dolo 650, 10 units total (5 to Rahul + 5 to Sneha)

-- Q10. Customer Lifetime Value: customers who spent more than 1000 in total.

```
SELECT c.name, SUM(o.total_amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
```

```
HAVING SUM(o.total_amount) > 1000;
-- ANSWER: Sneha Reddy - 1200.00
--         (Rahul Sharma totals 950.00, just under the 1000 threshold)

-- Q11. Inventory Audit: medicine_name, stock_quantity, supplier_name
--         for medicines with less than 50 units left in stock.
SELECT m.name AS medicine_name, m.stock_quantity, s.supplier_name
FROM medicines m
JOIN suppliers s ON m.supplier_id = s.supplier_id
WHERE m.stock_quantity < 50;
-- ANSWER:
--   Benadryl    - 20 units - MedPlus Supply Chain
--   Insulin Pen - 5  units - MedPlus Supply Chain
```

QUERY BANK - NEW QUERIES (Q12-Q21)

```
-- Q12. Medicines expiring before 1 Jan 2025.
SELECT name, expiry_date
FROM medicines
WHERE expiry_date < '2025-01-01'
ORDER BY expiry_date;
-- ANSWER:
--   Insulin Pen  - 2024-03-10
--   Azithral 500 - 2024-05-20
--   Benadryl     - 2024-11-15

-- Q13. Which supplier supplies 'Insulin Pen'?
```

```sql
SELECT s.supplier_name, s.contact_person, s.phone
FROM suppliers s
JOIN medicines m ON s.supplier_id = m.supplier_id
WHERE m.name = 'Insulin Pen';
-- ANSWER: MedPlus Supply Chain - Suresh Nair - 040-987654

-- Q14. Average order value (overall).
SELECT ROUND(AVG(total_amount), 2) AS avg_order_value FROM orders;
-- ANSWER: 487.00   ( (150+240+45+800+1200) / 5 )

-- Q15. Customers who have never placed an order.
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
-- ANSWER: Vikram Singh (the only customer with zero orders)

-- Q16. Total quantity sold per medicine (including medicines with 0 sales).
SELECT m.name, COALESCE(SUM(oi.quantity), 0) AS total_qty_sold
FROM medicines m
LEFT JOIN order_items oi ON m.medicine_id = oi.medicine_id
GROUP BY m.name
ORDER BY total_qty_sold DESC;
-- ANSWER:
--   Dolo 650     - 10
--   Azithral 500 - 4
--   Insulin Pen  - 2
--   Combiflam    - 1
```

```
--   Benadryl     - 0   (never ordered)


-- Q17. Second-highest order amount.
SELECT DISTINCT total_amount
FROM orders
ORDER BY total_amount DESC
LIMIT 1 OFFSET 1;
-- ANSWER: 800.00  (highest is 1200.00; second-highest is 800.00)


-- Q18. Number of line items per order.
SELECT order_id, COUNT(*) AS item_count
FROM order_items
GROUP BY order_id
ORDER BY order_id;
-- ANSWER:
--   order_id 1 : 1 item
--   order_id 2 : 1 item
--   order_id 3 : 1 item
--   order_id 4 : 1 item
--   order_id 5 : 3 items  (Insulin, Azithral, Dolo)


-- Q19. Medicines that have never been ordered.
SELECT m.name
FROM medicines m
LEFT JOIN order_items oi ON m.medicine_id = oi.medicine_id
WHERE oi.item_id IS NULL;
-- ANSWER: Benadryl
```

```
-- Q20. City-wise total revenue.
SELECT c.city, SUM(o.total_amount) AS city_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY city_revenue DESC;
-- ANSWER:
--   Hyderabad - 1200.00
--   Mumbai    - 950.00   (Rahul's two orders; Vikram has none)
--   Delhi     - 240.00
--   Ahmedabad - 45.00

-- Q21. Number of medicines supplied by each supplier (including suppliers with 0).
SELECT s.supplier_name, COUNT(m.medicine_id) AS medicine_count
FROM suppliers s
LEFT JOIN medicines m ON s.supplier_id = m.supplier_id
GROUP BY s.supplier_name
ORDER BY medicine_count DESC;
-- ANSWER:
--   Apollo Distributors  - 2  (Dolo 650, Azithral 500)
--   MedPlus Supply Chain - 2  (Benadryl, Insulin Pen)
--   Himalaya Wellness - 1  (Combiflam)
```

DEMO CALLS - Views / Stored Procedures / Triggers

```
-- Run these one at a time to see the objects from PART 4-6 in action.

-- Views:
```

```
-- SELECT * FROM view_order_details WHERE payment_mode = 'UPI';
-- SELECT * FROM view_low_stock;
-- SELECT * FROM view_customer_spend WHERE total_spent > 1000;
-- SELECT * FROM view_medicine_sales ORDER BY total_qty_sold DESC;

-- Stored Procedures:
-- CALL sp_get_customer_orders(1);            -- Rahul Sharma's orders
-- CALL sp_place_order(3, 1, 10, 'UPI');      -- Amit buys 10 Dolo 650 via UPI
-- CALL sp_revenue_by_mode();
-- CALL sp_restock_medicine(5, 20);           -- add 20 units to Insulin Pen

-- Triggers (fire automatically, nothing to CALL):
--   * Placing an order via sp_place_order, or any direct INSERT into
--         order_items, will automatically:
--         1) block the insert if stock is insufficient   -> trg_prevent_negative_stock
--         2) reduce medicines.stock_quantity             -> trg_reduce_stock
--         3) write a 'SALE' row into stock_log           -> trg_reduce_stock
--   * Calling sp_restock_medicine(...), or any direct UPDATE that raises
--         stock_quantity, will automatically write a 'RESTOCK' row into
--         stock_log                                      -> trg_restock_log
--   * Trying to DELETE FROM suppliers a supplier that still has
--         medicines linked to it will be blocked         -> trg_block_supplier_delete

-- Check the audit trail after running the demo calls above:
-- SELECT * FROM stock_log ORDER BY log_id;

-- ================================================================
-- END OF PROJECT SCRIPT
```

```
-- ========================================================================
```